Backtesting Model Report

1. Executive summary

This document formally reports on a machine-learning trading model implemented in the notebook `backtestingproject123.ipynb`. The model predicts short-term directional moves for individual instruments (labels: **BUY / HOLD / SELL**) using technical indicators derived from price time series. The primary model is a Random Forest classifier trained inside a scikit-learn `Pipeline` and persisted to `rf_stock_model_numeric.pkl`.

Key points: - Model type: `RandomForestClassifier` in a `Pipeline` with `StandardScaler`. - Training split: time-based (70% train, 30% test by timestamp quantile). - Labeling: multi-class labels (`-1`, `0`, `1`) produced by a horizon/shreshold rule (default `horizon=5`, `up_thresh=+2%`, `down_thresh=-2%`). - Feature set: moving averages, EMAs, MACD, RSI, rolling volatility, returns, and any numeric columns from the dataset after removing identifier and future columns.

Important limitation: I inspected the notebook code but did **not** execute the full pipeline because the required dataset (`project2_trading.csv`) was not available in the environment. Where the notebook contained placeholder `...` I inferred likely behavior from surrounding code; those inferences are noted below.

---

2. Notebook locations of core components

- **Cell 37** — `prepare_features(prices)`: constructs base features and a simple forward-return binary `target` in one variant.
- **Cell 38** — training pipeline: reads `project2_trading.csv`, defines `add_technical_indicators(...)`, `make_labels(...)`, sets `numeric_features`, performs time-based train/test split, fits `Pipeline(StandardScaler, RandomForestClassifier)`, prints classification report & confusion matrix, prints RF feature importances, and saves the pipeline via `joblib.dump(...)`.
- **Inference cells (around 42–44)** — build `stock_df` for a chosen `instrument_id`, compute features, create `X_new`, call `pipeline.predict(X_new)`, map predictions to `signal_text` (`-1:SELL, 0:HOLD, 1:BUY`) and print results.

---

3. Data & preprocessing

Data source: `project2_trading.csv` (expected columns: price/close, timestamp/date/time, instrument_id; additional numeric columns allowed).

Preprocessing steps observed: 1. Detect `price_col` automatically among `['close','price','last','close_price']` and detect the time column by name containing `timestamp|date|time`. 2. Convert time column to `datetime` and sort ascending. 3. Feature functions apply rolling windows with `min_periods=1` for early availability. Some functions drop NaNs after creation (`df.dropna()` in `prepare_features`) while inference uses `fillna(method='ffill').fillna(0)`. 4. Numeric features are selected by `select_dtypes(include=[np.number])` and certain columns are removed: `['future_close','future_return','label','trade_id','id','instrument_id']`.

Recommendation: align `dropna` / `fillna` policies between training and inference to avoid distributional mismatch.

---

4. Feature engineering (explicit)

The notebook explicitly computes the following features (non-exhaustive list): - `ma5`, `ma10`, `ma20` (simple moving averages) - `sma_fast` (5), `sma_slow` (20) - `ema_12`, `ema_26`, `macd = ema_12 - ema_26` - `volatility_10` (rolling std of 1-day returns) - `return` and `return_1` (pct changes) - RSI computed via EWMA of up/down moves - If present: `vol_ma_10`, `vol_ma_50`

Additionally, `numeric_features` will automatically include other numeric columns present in the CSV after removing identifiers and label columns.

---

5. Labeling scheme

The `make_labels(df, price_col='price', horizon=5, up_thresh=0.02, down_thresh=-0.02)` function creates labels by shifting forward `horizon` periods and computing `future_return`. The label mapping is inferred as: - `1` (BUY) if `future_return > up_thresh` - `-1` (SELL) if `future_return < down_thresh` - `0` (HOLD) otherwise

Note: a different `prepare_features` variant creates a binary `target = (fwd_return > 0).astype(int)`; confirm which labeling function you intend to use in final training.

---

6. Model architecture & training details

- **Pipeline:** `Pipeline([('scaler', StandardScaler()), ('rf', RandomForestClassifier(n_estimators=150, random_state=42, n_jobs=-1))])`.
- **Train/Test split:** time-based cutoff `cutoff_time = time_index.quantile(0.7)`; training data `time_index <= cutoff_time`.
- **Fit:** `pipeline.fit(X_train, y_train)`.

- **Evaluation printed:** `classification_report(y_test, y_pred)` and `confusion_matrix(y_test, y_pred, labels=[-1,0,1])`.
- **Feature importances:** `pipeline.named_steps['rf'].feature_importances_` displayed as top-10.
- **Model persistence:** saved with `joblib.dump(pipeline, MODEL_PATH)`.

Observations: - `StandardScaler` is used though Random Forests are scale-invariant (harmless but unnecessary). Keep the scaler if you plan to try models that need scaling. - No hyperparameter tuning nor time-series cross-validation visible in the training cell.

---

7. Evaluation & missing numeric results

The notebook prints standard classification metrics (precision, recall, f1) and a confusion matrix, but because the dataset was not available in the execution environment, no numeric evaluation values are reproduced in this report. To evaluate trading value, perform a backtest that maps model signals to trades and compute: - Cumulative return and annualized return - Annualized volatility - Sharpe ratio (use risk-free ~0 or appropriate local benchmark) - Maximum drawdown - Turnover and cost-sensitive returns (fees, slippage)

Recommendation: prefer financial metrics (Sharpe, CAGR, max drawdown) for model selection rather than classification accuracy alone.

---

8. Risks, assumptions & limitations

1. **Data absence**: `project2_trading.csv` not available for execution; parts of the notebook contain ellipses (`...`) that required inference.
2. **Potential leakage**: automatic inclusion of numeric columns requires a strict check that no future-derived columns are in `numeric_features`.
3. **Class imbalance**: likely — HOLD class may dominate; no mitigation observed.
4. **Single-split evaluation**: use of a single 70/30 split risks optimistic/unstable performance estimates.
5. **No production-level backtest**: training evaluation prints classification metrics but lacks an integrated backtesting system accounting for execution costs.

---

9. Recommendations (prioritized)

Immediate (must-do) - Confirm `numeric_features` excludes all future-derived columns and identifiers via asserts. - Print `y_train.value_counts()` and `y_test.value_counts()` to check class balance. - Convert predicted signals into an explicit backtest (apply fees = `0.0005` and realistic slippage) and report Sharpe, CAGR, and max drawdown.

Short-to-medium term - Implement time-series cross-validation (walk-forward) and use it for hyperparameter tuning. - Use `predict_proba` to filter low-confidence signals and reduce turnover. - Consider class weighting or resampling if imbalance harms learning. - Add SHAP explainability analysis.

Longer term / production - Monitoring for distribution drift, automated retraining triggers, and logging of live predictions vs. realized returns. - Integrate execution model (slippage, latency) into backtester and run stress tests.

---

10. Reproducibility checklist (to include alongside saved model)

- Model file: `rf_stock_model_numeric.pkl`
- Training dataset snapshot (CSV used for training) and version/hash
- Notebook version: `backtestingproject123.ipynb` (current)
- Training date range and timestamps for train/test split
- Exact feature list and order (store `numeric_features` list)
- Labeling parameters: `horizon`, `up_thresh`, `down_thresh`
- Random seed used (`random_state=42`)

---

11. Suggested next actions for me (choose one)

1. **Run full end-to-end execution** (requires upload of `project2_trading.csv`) to produce the exact classification report, confusion matrix, feature importances, and a single-run strategy backtest with P&L and Sharpe.
2. **Create a time-series cross-validation & hyperparameter tuning cell** and a robust backtest that includes fees/slippage and outputs financial metrics across folds.
3. **Produce a downloadable PDF** of this report.

Please tell me which option you prefer (1, 2 or 3), or request edits to this report. If you want option 1, upload the `project2_trading.csv` file and I will run it and append results to the report.

---

12. Appendix: quick validation snippets

(Insert into the notebook after training to validate assumptions)

```
# ensure no leakage
assert all(col not in numeric_features for col in
['future_close','future_return','label','trade_id','id','instrument_id'])

# check class balance
print('Train label distribution:', y_train.value_counts())
```

```
print('Test label distribution:', y_test.value_counts())

# sample feature importances
fi = pd.Series(pipeline.named_steps['rf'].feature_importances_,
index=numeric_features).sort_values(ascending=False)
print(fi.head(20))
```

End of report.